

GESTÃO DE CLUSTERS,
SEGURANÇA QUÂNTICA E
COLABORAÇÃO MULTIAGENTE

*VER E
COMPREENDER:
SEGMENTAÇÃO E
DETECÇÃO
INTELIGENTE DE
OBJETOS*



LISTA DE FIGURAS

Figura 1 - Bounding Box.....	9
Figura 2 - Intersection over Union (IoU)	10
Figura 3 - Bounding Boxes encontradas pelo modelo de detecção de objetos (esquerda). Bouding box resultante após aplicação de NMS (direita).....	11
Figura 4 - R-CNN	12
Figura 5 - Resposta do modelo R-CNN.....	15
Figura 6 - YOLO	17
Figura 7 - Resultado da Segmentação Semântica	21
Figura 8 - Segmentação Instanciada com Mask R-CNN.....	23
Figura 9 - Comparação entre tipos de segmentação	24
Figura 10 - Algoritmo DeepSort.....	28
Figura 11 - Saída do Algoritmo de Object Tracking para um frame	30
Figura 12 - Algoritmo SAM2	33
Figura 13 - Saída do algoritmo SAM2	36
Figura 14 - Máscaras resultantes do SAM2	37

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Usando uma RCNN pré-treinada.....	14
Código-fonte 2 - Usando um modelo de segmentação de imagens pré-treinado.....	21
Código-fonte 3 - Object Tracking.....	30
Código-fonte 4 - Setup do SAM2.....	34
Código-fonte 5 - Segmentação de imagem com SAM2	35
Código-fonte 6 - Máscaras encontradas pelo SAM2	36

EMANIP

SUMÁRIO

1 VER E COMPREENDER: SEGMENTAÇÃO E DETECÇÃO INTELIGENTE DE OBJETOS.....	5
1.1 Introdução à Detecção e Segmentação de Objetos	5
1.2 Evolução histórica	6
1.3 Casos de uso reais.....	7
2 FUNDAMENTOS E REPRESENTAÇÕES MATEMÁTICAS	8
2.1 Bounding Boxes	8
2.2 Métricas de Avaliação	9
2.3 Non-Maximum Suppression	10
3 MODELOS E ARQUITETURAS DE DETECÇÃO DE OBJETOS.....	12
3.1 Arquiteturas de duas etapas (Two-Stage Detectors).....	12
3.2 Arquiteturas de uma etapa (One-Stage Detectors)	15
3.3 Comparação entre modelos	17
4 SEGMENTAÇÃO DE IMAGENS: CONCEITOS E ARQUITETURAS	19
4.1 Segmentação Semântica	19
4.2 Segmentação Instanciada	22
4.3 Segmentação Panóptica	23
5 DETECÇÃO E SEGMENTAÇÃO EM VÍDEO	25
5.1 Desafios Específicos de Vídeo	25
5.2 Abordagens para Vídeo.....	26
5.3 Rastreamento de Objetos.....	27
5.4 Aplicações em Tempo Real.....	30
6 SEGMENT ANYTHING MODEL 2 (SAM 2): UMA NOVA GERAÇÃO DE MODELOS DE SEGMENTAÇÃO.....	32
7 LIMITAÇÕES, DESAFIOS ÉTICOS E TENDÊNCIAS FUTURAS	38
7.1 Limitações Técnicas	38
7.2 Questões Éticas	39
7.3 Tendências Futuras.....	40
CONCLUSÃO.....	42
REFERÊNCIAS.....	44

1 VER E COMPREENDER: SEGMENTAÇÃO E DETECÇÃO INTELIGENTE DE OBJETOS

1.1 Introdução à Detecção e Segmentação de Objetos

Em Visão Computacional, tarefas básicas como **classificação de imagens** concentram-se em responder à pergunta “o que há nesta imagem?”. Isso significa atribuir uma ou mais classes a uma imagem inteira sem considerar onde os objetos se encontram ou quanto espaço ocupam no quadro visual. Por exemplo, um modelo de classificação pode receber uma foto e indicar que ela contém um gato ou um carro com base nas características gerais extraídas dos pixels. Essa abordagem é fundamental, mas insuficiente para problemas que exigem compreensão espacial da cena.

A **detecção de objetos** estende a classificação ao incluir a **localização** dos objetos dentro da imagem. Em vez de simplesmente reconhecer que existe um carro, a detecção localiza esse carro desenhando uma “caixa delimitadora” (bounding box) que indica sua posição e tamanho. Essa mudança de foco é crucial para tarefas que dependem de saber não apenas o que está presente, mas onde cada objeto está.

Já a **segmentação semântica** divide a imagem em regiões significativas e rotula cada pixel de acordo com sua classe. Diferentemente da detecção, ela não retorna caixas delimitadoras, mas sim uma máscara detalhada que cobre toda a área ocupada por classes de interesse. Essa abordagem é especialmente útil quando a forma e a extensão dos objetos são importantes, como em imagens médicas ou cartografia automatizada.

A **segmentação instanciada** combina os objetivos da detecção e da segmentação semântica ao distinguir cada instância individual de objetos pertencentes à mesma classe. Por exemplo, em uma cena com três carros, a segmentação semântica rotula todos os pixels dos carros com o mesmo rótulo “carro”. A segmentação instanciada, por outro lado, identifica e separa as três instâncias distintas, atribuindo máscaras específicas a cada uma delas.

A limitação da **classificação global** torna-se evidente quando se tenta aplicar esses modelos em cenários reais que exigem decisões espaciais detalhadas. Por

exemplo, um algoritmo que apenas classifica imagens como “contendo pedestres” não fornece a localização dos pedestres nem possibilita ações específicas como travar freios ou estimar distâncias; por isso, a tarefa de **localizar** é tão importante quanto a de **reconhecer**, principalmente quando se busca replicar capacidades perceptivas humanas em sistemas automatizados.

1.2 Evolução histórica

Antes da popularização das técnicas de Deep Learning, a detecção de objetos e outras tarefas de entendimento visual dependiam intensamente de métodos baseados em extração manual de características e classificadores tradicionais. Uma dessas abordagens clássicas empregava janelas deslizantes (*sliding window*) que percorriam toda a imagem em múltiplas escalas, verificando em cada posição se havia ou não um objeto de interesse. Essa estratégia gerava uma grande quantidade de sub-imagens que deveriam ser analisadas individualmente, o que tornava o processo computacionalmente caro e sensível a variações de escala e posição.

O uso de HOG (Histogram of Oriented Gradients) e SVM (Support Vector Machines) representou um avanço importante nesta época. HOG descreve regiões da imagem por histogramas de gradientes orientados, capturando contornos e formas locais que eram usados como entradas para classificadores como SVM. Essa combinação demonstrou excelente desempenho, por exemplo, na detecção de pedestres em imagens estáticas, especialmente quando combinada com modelos de escala multiescala.

Outro marco histórico é o algoritmo de Viola-Jones, proposto no início dos anos 2000. Inicialmente concebido para detecção de faces, ele utiliza características tipo Haar e um classificador em cascata treinado por boosting. Embora hoje seja superado por modelos baseados em redes neurais em termos de precisão, ele foi um dos primeiros métodos realmente capazes de realizar detecção em “tempo real” sob restrições computacionais modestas.

Com o avanço do poder de processamento e o desenvolvimento de grandes conjuntos de dados anotados, como ImageNet, PASCAL VOC e MS COCO, o campo de Visão Computacional passou por uma transformação dramática. Os dados

rotulados em larga escala permitiram treinar redes neurais profundas capazes de aprender representações visuais diretamente dos pixels, substituindo a necessidade de extração manual de características e estabelecendo bases para os modelos modernos de detecção e segmentação.

1.3 Casos de uso reais

Os algoritmos de detecção e segmentação de objetos se tornaram fundamentais em muitas áreas de aplicação prática. Em veículos autônomos, por exemplo, é necessário identificar com precisão a posição de pedestres, outros veículos, faixas de trânsito e obstáculos para permitir decisões de direção seguras. A interpretação visual neste contexto precisa ser contínua e em tempo real para responder a cenários dinâmicos.

No contexto do varejo inteligente, técnicas de Visão Computacional são empregadas para monitorar produtos em prateleiras, rastrear o movimento de clientes e automatizar processos como lançamentos de compras sem checkout tradicional. A detecção de objetos integrada com sensores adicionais torna possível a identificação de itens retirados ou reposicionados em um estabelecimento físico.

Na saúde, imagens médicas como tomografias e ressonâncias magnéticas podem ser segmentadas automaticamente para destacar regiões anatômicas ou áreas patológicas. A segmentação semântica permite que cada pixel seja interpretado de acordo com seu contexto clínico, facilitando diagnósticos assistidos por máquina.

Em segurança e vigilância, sistemas equipados com câmeras inteligentes podem detectar e rastrear pessoas ou veículos suspeitos, alertando operadores ou integrando-se a sistemas de resposta automática. Já na indústria e robótica, a compreensão visual é usada para inspecionar produtos em linhas de produção, detectar defeitos ou orientar braços robóticos para manipulação precisa de objetos.

2 FUNDAMENTOS E REPRESENTAÇÕES MATEMÁTICAS

2.1 Bounding Boxes

Na tarefa de detecção de objetos, um conceito central é o de **bounding box**, ou caixa delimitadora, que representa a posição de um objeto detectado dentro de uma imagem. Essa caixa é essencialmente um retângulo que circunda aproximada e geometricamente o objeto de interesse, permitindo tanto localizar sua presença quanto estimar sua ocupação espacial no quadro visual. Existem duas representações principais para descrever bounding boxes em termos numéricos. A primeira representa a caixa utilizando as coordenadas (x, y, w, h) , em que (x, y) denota o ponto no canto superior esquerdo da caixa, enquanto w e h representam sua largura e altura, respectivamente. A segunda representação usada com frequência é (x_1, y_1, x_2, y_2) , em que (x_1, y_1) é o canto superior esquerdo e (x_2, y_2) é o canto inferior direito da caixa. Ambas as representações podem ser convertidas entre si por meio de operações aritméticas simples, e a escolha entre uma ou outra depende de convenções da ferramenta ou framework de implementação.

A normalização dos valores das coordenadas é uma prática comum quando se trabalha com redes neurais profundas: em vez de usar valores absolutos de pixels (por exemplo, 0 a 1023 em uma imagem 1024×1024), normalizamos os valores para o intervalo de $[0,1]$ dividindo pela largura e altura da imagem. Isso ajuda a rede a aprender parâmetros que não dependem da resolução exata dos dados de entrada.

Transformações geométricas são frequentemente aplicadas durante treinamento e pré-processamento. Essas transformações incluem escalonamento (*scaling*), rotação e recorte (*cropping*). Cada transformação altera as posições e/ou dimensões da bounding box original, sendo necessário recalcular as coordenadas conforme a transformação. Por exemplo, ao escalar uma imagem pela metade, todas as coordenadas X e Y das caixas devem ser igualmente reduzidas. Isso é essencial para manter a consistência entre os objetos reais e as suas representações em diferentes versões transformadas da imagem.

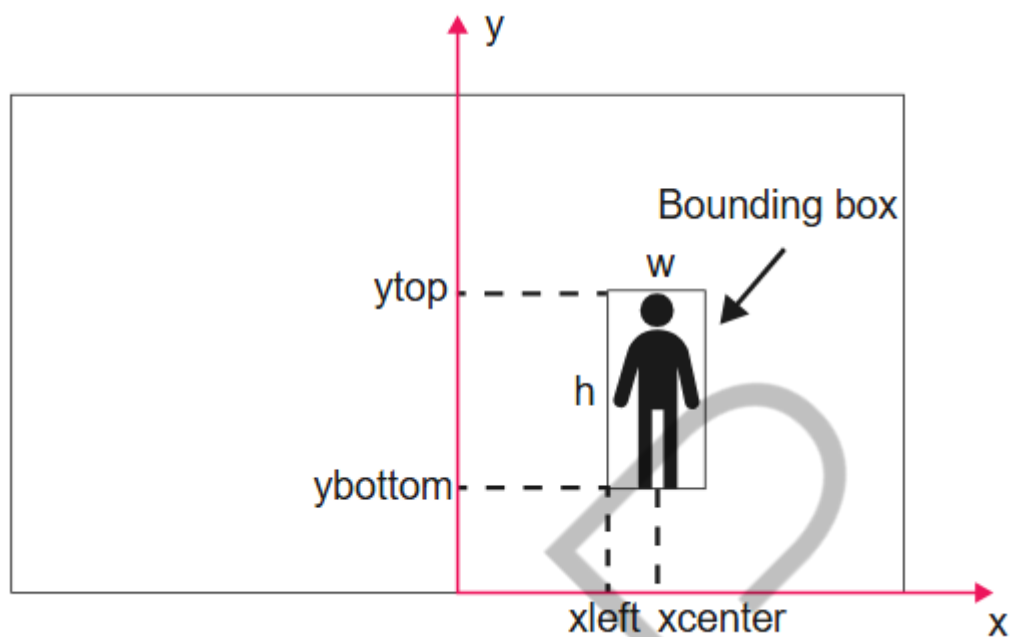


Figura 1 - Bounding Box

Fonte: UAV-based crowd surveillance in post covid-19 era (2021), adaptado por FIAP (2026)

2.2 Métricas de Avaliação

Uma vez que um modelo de detecção produz previsões de bounding boxes, é fundamental avaliar o quão bem essas previsões correspondem às posições verdadeiras dos objetos anotados (denominadas ground truth). A métrica mais elementar para avaliar essa correspondência é o Intersection over Union (IoU), ou Interseção sobre União, que mede a razão entre a área de interseção das caixas previstas e verdadeiras e a área de sua união. Formalmente, essa medida é calculada pela razão entre a área de interseção e a área de união das duas caixas, resultando em um valor real entre 0 e 1, em que 1 representa um alinhamento perfeito entre a previsão e o ground truth. Quanto maior for o valor de IoU, melhor a sobreposição e, portanto, a precisão espacial do detector.

A partir de IoU, podemos definir as métricas tradicionais de classificação adaptadas para detecção de objetos. Precision, ou precisão, mede a proporção de previsões corretas entre todas as previsões feitas pelo modelo e é definida em termos de verdadeiros positivos (TP) e falsos positivos (FP). Recall, ou sensibilidade, mede a capacidade do modelo de encontrar todas as instâncias reais de um objeto, ou seja, a proporção de verdadeiros positivos em relação ao total de objetos presentes (ground truth), contabilizando falsos negativos (FN). A métrica F1-score é a média harmônica

entre precisão e recall e oferece um único valor que equilibra erros de omissão e comissões, sendo especialmente útil quando há *trade-off* entre esses dois aspectos.

Além dessas, Average Precision (AP) é uma métrica que resume a relação precisão-recall ao calcular a área sob a curva de precisão *versus* recall para uma classe específica, variando o limiar de confiança das predições. Mean Average Precision (mAP), por sua vez, gera um valor agregado ao tirar a média das APs de todas as classes presentes no conjunto de dados avaliado. Essa métrica é amplamente utilizada para comparar modelos em desafios como PASCAL VOC e MS COCO e reflete tanto a habilidade de localizar objetos quanto de classificá-los corretamente em múltiplas classes.

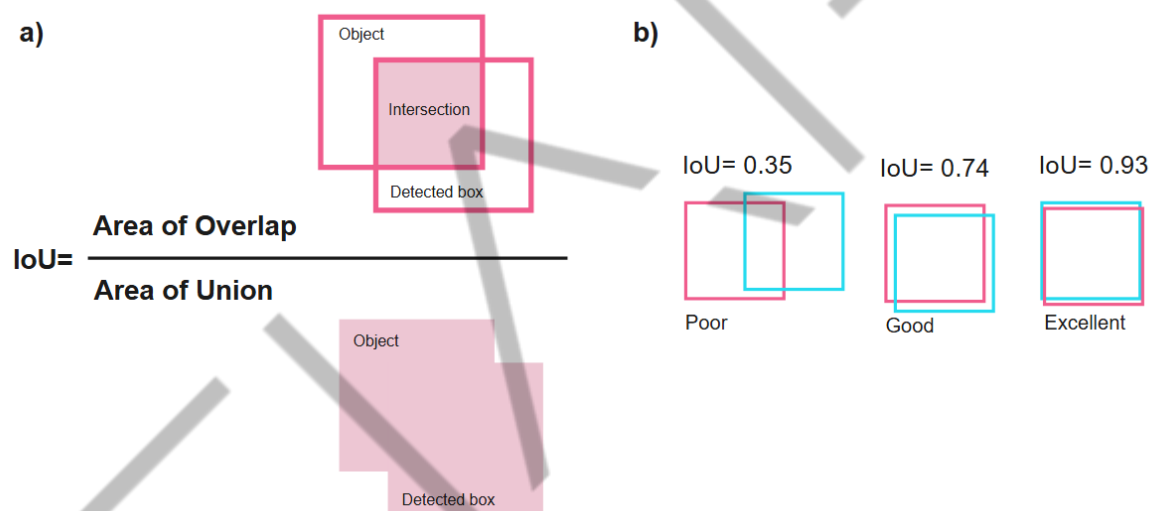


Figura 2 - Intersection over Union (IoU)

Fonte: Loss Functions and Metrics in Deep Learning. A Review (2023), adaptado por FIAP (2026)

2.3 Non-Maximum Suppression

Mesmo com métricas bem definidas para avaliar a qualidade das detecções, outro desafio que aparece na prática é o fato de um modelo poder gerar múltiplas caixas preditas para o mesmo objeto real. Isso ocorre porque, em muitas arquiteturas modernas de detecção, como YOLO ou Faster R-CNN, diversas regiões candidatas podem ser propostas com pontuações elevadas para a mesma instância de objeto. Para determinar uma única caixa representativa por objeto, utiliza-se um procedimento chamado Non-Maximum Suppression (NMS).

O NMS funciona ao ordenar todas as bounding boxes previstas por uma medida de confiança (score) decrescente. Em seguida, a caixa com o maior score é selecionada como detecção final, e todas as caixas com IoU acima de um determinado limiar em relação a essa caixa são descartadas. O algoritmo repete esse processo até que todas as caixas tenham sido processadas, resultando em um conjunto filtrado de detecções sem redundância múltipla sobre o mesmo objeto real.

Uma limitação do NMS tradicional é que ele pode, em situações de objetos muito próximos, eliminar predições úteis devido ao critério rígido de limiar. Para mitigar isso, variantes como o Soft-NMS foram propostas, em que as caixas não são imediatamente descartadas quando sua IoU ultrapassa o limiar, mas sim têm sua pontuação suavemente reduzida conforme sua sobreposição aumenta. Essa abordagem mantém o trade-off entre precisão e recall, podendo resultar em melhor desempenho geral.

O impacto do NMS é diretamente percebido no desempenho final: um ajuste muito agressivo do limiar pode aumentar a precisão (eliminando falsos positivos redundantes), mas reduzir o recall (ao descartar predições úteis). Por outro lado, um limiar mais flexível pode manter mais detecções válidas, ao custo de maior redundância. Esse tipo de escolha depende fortemente do cenário de aplicação, por exemplo, em sistemas de vigilância em tempo real *versus* análises offline de alta precisão.



Figura 3 - Bounding Boxes encontradas pelo modelo de detecção de objetos (esquerda). Bounding box resultante após aplicação de NMS (direita)

Fonte: The Python Code (2024)

3 MODELOS E ARQUITETURAS DE DETECÇÃO DE OBJETOS

A detecção de objetos em Visão Computacional evoluiu a partir de modelos que combinam reconhecimento de padrões com estimativas precisas de localização, traduzindo imagens em listas estruturadas de objetos com suas posições e classes. Modelos avançados para essa tarefa podem ser agrupados em arquiteturas de duas etapas (two-stage detectors) e de uma etapa (one-stage detectors). Essa divisão reflete escolhas arquitetônicas que influenciam diretamente a precisão, a velocidade e a complexidade computacional dos sistemas de detecção. De modo geral, métodos de duas etapas, como Faster R-CNN, realizam proposições de regiões candidatas antes de classificá-las com maior profundidade, enquanto os modelos de uma etapa, como YOLO e SSD, realizam detecção diretamente em uma única passagem pela rede neural, priorizando velocidade de inferência.

3.1 Arquiteturas de duas etapas (Two-Stage Detectors)

Os detectores de duas etapas consolidaram-se como uma das abordagens mais robustas para detecção de objetos com alta precisão. Uma das representações mais conhecidas dessa família é o Faster R-CNN, que se diferencia por integrar em sua própria arquitetura uma rede que gera propostas de regiões candidatas, a Region Proposal Network (RPN). Antes do Faster R-CNN, os modelos R-CNN e Fast R-CNN dependiam de algoritmos externos de geração de propostas, como Selective Search, para sugerir regiões de interesse (Rois).

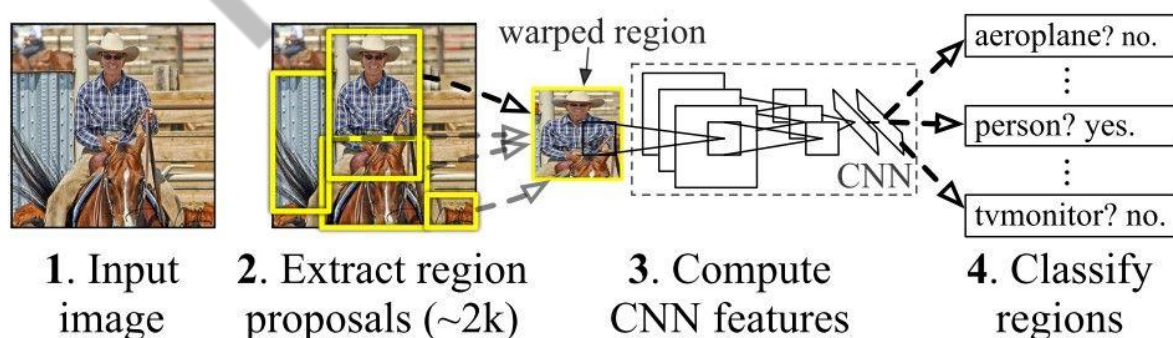


Figura 4 - R-CNN
Fonte: DigitalOcean (2025)

No Faster R-CNN, a Region Proposal Network é uma sub-rede completamente convolucional que, para cada posição do mapa de características extraído pelo

backbone, prediz um conjunto de possíveis caixas candidatas e escores de “objectness”, isto é, de probabilidade de um objeto estar presente naquela caixa. Essa característica torna o processo de geração de propostas mais eficiente e integrado, eliminando a necessidade de etapas externas e permitindo treinamento end-to-end.

O backbone refere-se ao extrator de características profundas usado pelo modelo para processar a imagem de entrada. Architecturas como ResNet e VGG são exemplos comuns de backbones que fornecem mapas de características ricas e discriminativas que alimentam tanto a RPN quanto as camadas de classificação e regressão da rede. Essas redes pré-treinadas em grandes conjuntos de dados de classificação, como ImageNet, aprendem representações visuais úteis que aceleram a convergência e melhoram o desempenho do detector.

Uma etapa técnica importante no fluxo do Faster R-CNN é o processo de ROI Pooling (ou sua variante mais precisa, ROI Align), responsável por extrair um tamanho fixo de características para cada região candidata, independentemente do tamanho original da caixa. O ROI Pooling discretiza os mapas de características para uma dimensão fixa, mas pode introduzir pequenas imprecisões devido ao arredondamento de coordenadas; o ROI Align, por outro lado, utiliza amostragem bilinear para alinhar melhor os valores de características com as coordenadas originais, reduzindo erros de localização.

O fluxo completo de treinamento e inferência de um modelo como o Faster R-CNN começa com a passagem da imagem pelo backbone, seguida da geração de propostas pela RPN. Cada proposta é então refinada e classificada por head(s) específicas que retornam classes de objeto e boxes ajustados. O treinamento alterna a atualização dos pesos do backbone, da RPN e do head final, geralmente com funções de perda que combinam regressão de caixa e classificação de objeto.

Em termos práticos de implementação, frameworks como PyTorch e TensorFlow disponibilizam módulos de Faster R-CNN pré-treinados que podem ser finamente ajustados em novos conjuntos de dados. Um exemplo simples em PyTorch, ilustrando a criação de um modelo pré-treinado pronto para inferência, seria:

```
1 | import requests
2 | from PIL import Image
3 | import io
4 | import torchvision.transforms as T
5 | import torchvision
6 | import torch
7 |
8 | # URL de uma imagem de exemplo
9 | image_url = "https://quatorrodas.abril.com.br/wp-
content/uploads/2024/02/FiatMobiLikeMY245.jpg" # Exemplo de
imagem de domínio público
10 |
11 | # Baixa a imagem da URL
12 | response = requests.get(image_url)
13 | response.raise_for_status() # Levanta um erro para
códigos de status HTTP ruins (4xx ou 5xx)
14 |
15 | # Abre a imagem usando PIL (Pillow)
16 | image_pil =
Image.open(io.BytesIO(response.content)).convert("RGB") #
Converte para RGB se necessário
17 |
18 | # Define transformações para converter a imagem PIL em
um tensor PyTorch
19 | transform = T.Compose([
20 |     T.Resize((800, 800)), # Redimensiona a imagem para o
tamanho esperado pelo modelo
21 |     T.ToTensor()          # Converte a imagem PIL em um
tensor PyTorch (C, H, W)
22 | ])
23 |
24 | image_tensor_from_url = transform(image_pil)
25 |
26 |
27 | model =
torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrai
ned=True)
28 | model.eval()
29 |
30 | outputs_from_url = model([image_tensor_from_url])
31 | print(outputs_from_url)
32 |
33 |
```

Código-fonte 1 - Usando uma RCNN pré-treinada
Fonte: Elaborado pelo autor (2025)

O código anterior ilustra a inferência de um modelo pré-treinado sem necessidade de treinamento adicional imediato, útil para aplicações mais simples que

utilizam detecção de objetos. Observe a seguir o bounding box resultando com a respectiva classe do objeto detectado.



Figura 5 - Resposta do modelo R-CNN
Fonte: Elaborado pelo autor (2025)

3.2 Arquiteturas de uma etapa (One-Stage Detectors)

Enquanto os detectores de duas etapas priorizam precisão por meio de processos subdivididos, os detectores de uma etapa combinam simultaneamente a localização de caixas e a classificação de objetos em uma única passada pela rede. Esse design tende a ser mais eficiente em termos de velocidade de inferência, o que os torna adequados para aplicações em tempo real ou embarcadas, embora tradicionalmente tenham oferecido precisão ligeiramente inferior à arquitetura de duas etapas.

O YOLO (You Only Look Once) é um dos primeiros e mais influentes detectores de uma etapa. Introduzido por Joseph Redmon e colaboradores, o YOLO reformula a detecção como um problema de regressão único, em que a rede neural convolucional prevê diretamente as caixas delimitadoras e as probabilidades de classe para cada célula de uma grade aplicada sobre a imagem. A ideia central é que, ao fazer isso em

uma única passada, o modelo é capaz de alcançar taxas de quadros por segundo (FPS, Frames Per Second) muito superiores às arquiteturas de duas etapas.

A evolução do YOLO passou por múltiplas versões, desde o YOLOv1, que estabeleceu esse paradigma de regressão, até versões modernas como YOLOv8 (e variantes que continuam a evoluir), que incorporam melhorias arquitetônicas, estruturas de backbone mais profundas e técnicas como anchor boxes e predição multi-escala.

Em contrapartida ao YOLO, o SSD (Single Shot Detector) também é um detector de passada única que foca em gerar previsões de objetos em múltiplas escalas. Ele utiliza mapas de características extraídos de diferentes camadas da rede convolucional para detectar objetos de diferentes tamanhos. O SSD emprega um conjunto de default boxes ou anchor boxes de diferentes formas e tamanhos e, para cada uma dessas caixas, prediz deslocamentos e escores de classe. Essa técnica permite que o SSD lide melhor com objetos de escalas variadas sem a necessidade de múltiplas passadas ou etapas complexas de proposição de regiões.

Do ponto de vista de desempenho, detectores de uma etapa como YOLO e SSD tendem a ter maior velocidade de processamento em comparação com arquiteturas de duas etapas, sendo capazes de operar em vídeos e aplicações em tempo real com restrições computacionais. Ao mesmo tempo, eles podem apresentar limitações em precisão espacial, especialmente para objetos pequenos ou densamente agrupados, onde as etapas adicionais de refinamento de propostas em modelos como Faster R-CNN podem capturar detalhes mais finos.

YOLO: You Only Look Once

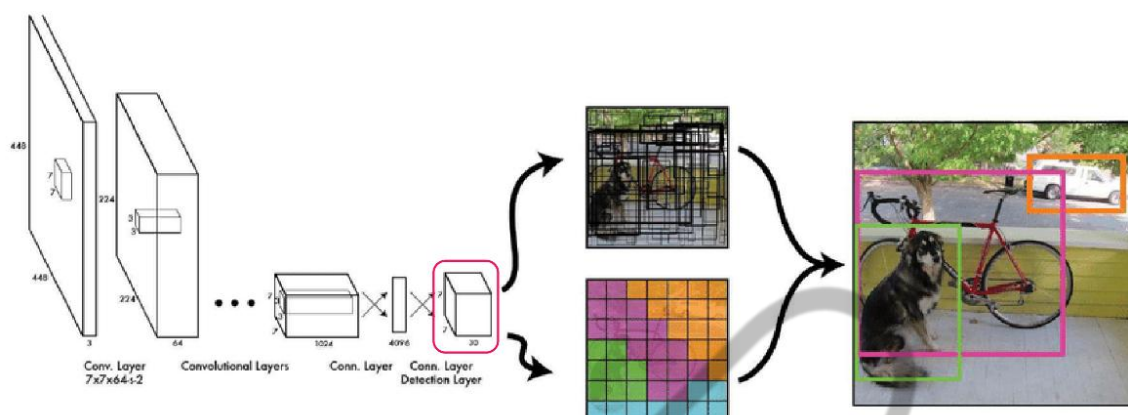


Figura 6 - YOLO

Fonte: Computer Vision: YOLO introduction (2023), adaptado por FIAP (2026)

3.3 Comparação entre modelos

Uma comparação prática entre esses modelos envolve múltiplos aspectos, incluindo velocidade de inferência, geralmente medida em quadros por segundo, precisão, frequentemente avaliada por métricas como mean Average Precision, e complexidade computacional, que influenciam as exigências de memória e tempo de treinamento.

Modelos como Faster R-CNN, devido às suas múltiplas fases de proposição e refinamento, tipicamente alcançam valores mais elevados de mAP em benchmarks padrão, como PASCAL VOC ou MS COCO, mas com menor FPS em hardware limitado. Em contraste, modelos como YOLO ou SSD sacrificam parte da precisão para alcançar taxas de processamento significativamente maiores, tornando-os escolhas frequentes em aplicações que exigem baixa latência, como detecção em vídeo ao vivo ou execução em dispositivos embarcados com recursos limitados.

A escolha do modelo adequado depende, portanto, do contexto de aplicação: cenários que priorizam precisão máxima de detecção (por exemplo, aplicações médicas críticas) podem se beneficiar de arquiteturas de duas etapas, enquanto aplicações em tempo real ou com restrições de hardware podem se beneficiar de detectores de uma etapa.

Esse conjunto de modelos representa a base atual para sistemas de detecção de objetos em Visão Computacional e serve como ponto de partida para tarefas mais avançadas, como segmentação instanciada e detecção em sequência temporal de vídeo. Considerações de desempenho, custo computacional e requisitos de aplicação guiam a escolha entre essas abordagens, e extensões modernas continuam a ampliar os limites de precisão e eficiência.



4 SEGMENTAÇÃO DE IMAGENS: CONCEITOS E ARQUITETURAS

A segmentação de imagens é uma tarefa essencial da visão computacional cujo objetivo é decompor uma imagem digital em regiões de significado visual ou semântico, de forma que cada região represente um objeto específico ou parte dele. Essa tarefa é realizada ao nível de pixels, atribuindo a cada pixel um rótulo que indica sua associação com determinada classe ou objeto. Essa abordagem difere de tarefas como detecção de objetos que operam com caixas delimitadoras (bounding boxes), pois a segmentação proporciona uma compreensão mais fina da estrutura interna de cada objeto e do contexto visual em que eles estão inseridos.

4.1 Segmentação Semântica

A segmentação semântica é definida como a tarefa de atribuir um rótulo de classe a cada pixel de uma imagem, independentemente de qual instância específica de objeto ele pertence. Nesse tipo de segmentação, todos os pixels que pertencem à mesma categoria recebem o mesmo rótulo; por exemplo, em uma imagem com vários carros, todos os pixels que compõem os carros são rotulados simplesmente como “carro”, sem distinção entre instâncias individuais. Essa representação ao nível pixel fornece uma imagem segmentada em que cada região pertence a uma classe, como “céu”, “estrada”, “pedestre” ou “vegetação”.

Na prática, modelos de segmentação semântica utilizam redes neurais profundas especificamente adaptadas para realizar classificação densa de pixels. Uma arquitetura clássica é a Fully Convolutional Network (FCN), que substitui as camadas totalmente conectadas por camadas convolucionais, permitindo que a rede produza mapas de saída com a mesma dimensão espacial do input. Outras arquiteturas populares incluem U-Net, SegNet e DeepLab, que incorporam mecanismos de upsampling (interpolação) para recuperar resoluções espaciais mais altas nas saídas.

Por exemplo, usando PyTorch e a biblioteca torchvision, é possível carregar um modelo de segmentação semântica pré-treinado e realizar inferência em uma imagem da seguinte forma:

```
1 | import requests
2 | import io
3 | import torch
4 | import torchvision
5 | import torchvision.transforms as T
6 | import matplotlib.pyplot as plt
7 | import numpy as np
8 | from PIL import Image
9 |
10|
11| model =
torchvision.models.segmentation.fcn_resnet50(pretrained=True
)
12| model.eval()
13|
14|
15| image_url = "https://www.autoracing.com.br/wp-
content/uploads/2014/12/Chevrolet-Onix.jpg"
16|
17| response = requests.get(image_url)
18| response.raise_for_status()
19|
20| image =
Image.open(io.BytesIO(response.content)).convert("RGB")
21|
22| image_np = np.array(image)
23|
24|
25| transform = T.Compose([
26|     T.Resize(256),
27|     T.ToTensor(),
28|     T.Normalize(
29|         mean=[0.485, 0.456, 0.406],
30|         std=[0.229, 0.224, 0.225]
31|     )
32| ])
33|
34| input_tensor = transform(image).unsqueeze(0)
35|
36|
37| with torch.no_grad():
38|     output = model(input_tensor)["out"][0]
39|
40| # Pixel-wise class prediction
41| pred_mask = output.argmax(0).cpu().numpy()
42|
43|
44| fig, axes = plt.subplots(1, 2, figsize=(16, 8))
45|
46| # Original image
```

```
47| axes[0].imshow(image_np)
48| axes[0].set_title("Original Image")
49| axes[0].axis("off")
50|
51| # Segmented image
52| axes[1].imshow(pred_mask, cmap="tab20")
53| axes[1].set_title("Semantic Segmentation (FCN-
ResNet50)")
54| axes[1].axis("off")
55|
56| plt.tight_layout()
57| plt.show()
58|
```

Código-fonte 2 - Usando um modelo de segmentação de imagens pré-treinado
Fonte: Elaborado pelo autor (2025)

Esse exemplo demonstra como executar segmentação semântica usando um modelo pré-treinado, retornando uma máscara em que cada pixel é classificado semanticamente em uma classe. Podemos verificar o resultado na figura a seguir:

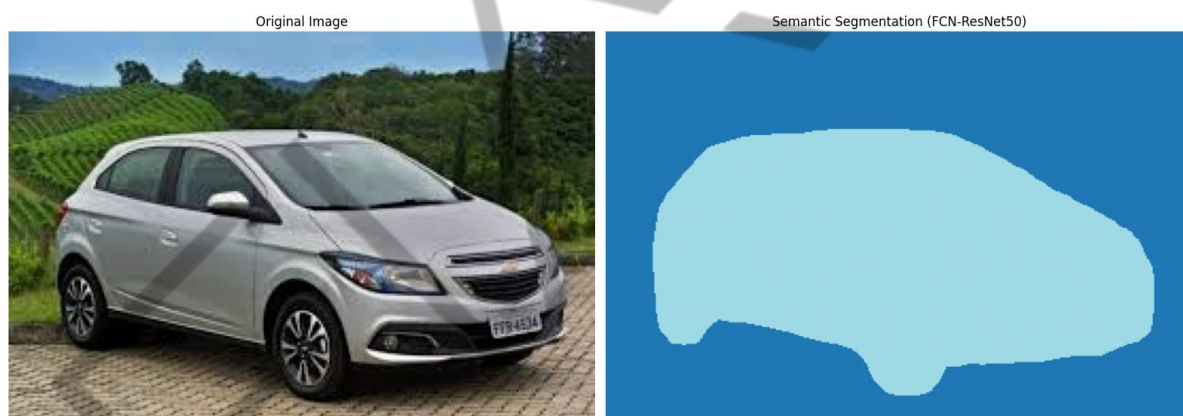


Figura 7 - Resultado da Segmentação Semântica
Fonte: Elaborado pelo autor (2025)

As aplicações da segmentação semântica são variadas. Em veículos autônomos, por exemplo, a segmentação ajuda a separar pistas de rodagem de objetos móveis como pedestres ou outros carros. Em imagens médicas, permite delinear regiões anatômicas ou patológicas em exames, como tumores ou estruturas orgânicas específicas. Em análise de imagens de satélite, a segmentação semântica pode separar regiões de vegetação, água e edificações.

4.2 Segmentação Instanciada

A segmentação instanciada amplia a segmentação semântica ao diferenciar instâncias individuais de objetos, mesmo quando estes pertencem à mesma classe. Enquanto a segmentação semântica rotula todos os pixels de uma classe com o mesmo rótulo (por exemplo, todos os carros ficam juntos em uma “mancha” segmentada), a segmentação instanciada fornece máscaras individuais para cada objeto distinto, por exemplo, “carro 1”, “carro 2” e assim por diante. Isso é especialmente útil em tarefas onde é essencial contar e identificar cada instância de objeto separadamente.

A segmentação instanciada está mais próxima da detecção de objetos tradicionais no que diz respeito à necessidade de separar objetos individuais, mas vai além ao delimitar precisamente seus contornos ao nível de pixel, em vez de utilizar apenas caixas delimitadoras. Um dos modelos mais conhecidos para segmentação instanciada é o Mask R-CNN, que estende arquiteturas de detecção de objetos como Faster R-CNN adicionando um ramo de predição de máscara que prevê a forma exata de cada instância dentro da região proposta.

A relação entre segmentação instanciada e detecção de objetos pode ser vista como complementar: a detecção fornece caixas delimitadoras que aproximam a posição dos objetos, enquanto a segmentação instanciada refina essa informação para produzir máscaras detalhadas que descrevem a forma completa de cada objeto. Essa distinção é fundamental em aplicações como manipulação robótica precisa, contagem de objetos em ambientes densos e análise de cenas complexas com múltiplos objetos próximos ou sobrepostos.

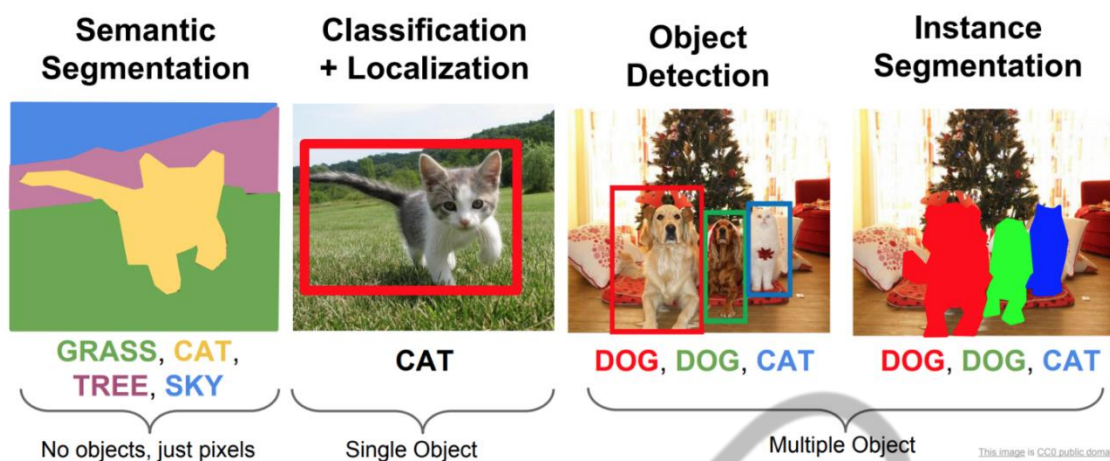


Figura 8 - Segmentação Instanciada com Mask R-CNN

Fonte: DataScience (2019)

4.3 Segmentação Panóptica

A segmentação panóptica surge como uma tarefa mais abrangente que unifica a segmentação semântica e a segmentação instanciada em um único framework. Nessa abordagem, cada pixel recebe tanto um rótulo de classe quanto uma identificação de instância, proporcionando uma compreensão completa da cena visual. Isso significa que todos os pixels são rotulados semanticamente e, ao mesmo tempo, aqueles que pertencem a objetos “contáveis” (como pessoas ou carros) recebem IDs que distinguem instâncias individuais, enquanto regiões amorfas (como céu ou estrada) são rotuladas apenas semanticamente.

A segmentação panóptica resolve limitações das abordagens anteriores. Na segmentação semântica, classes de objetos distintas podem ser reconhecidas, mas objetos individuais da mesma classe não são separados. Na segmentação instanciada, as regiões de fundo ou classes não contáveis são frequentemente tratadas apenas de forma agregada ou ignorada. A panóptica integra as duas abordagens para fornecer um mapa de saída único que contém informações completas sobre classe e identidade de instância para cada pixel.

Esse tipo de segmentação é particularmente útil em aplicações avançadas de entendimento de cena, como em veículos autônomos de alta precisão, onde não apenas a localização de objetos é crítica, mas também a compreensão detalhada das relações espaciais entre diferentes partes da cena. Outro uso relevante está na

robotização colaborativa, em que identificar claramente cada objeto em um ambiente compartilhado com humanos é essencial para segurança e eficiência.

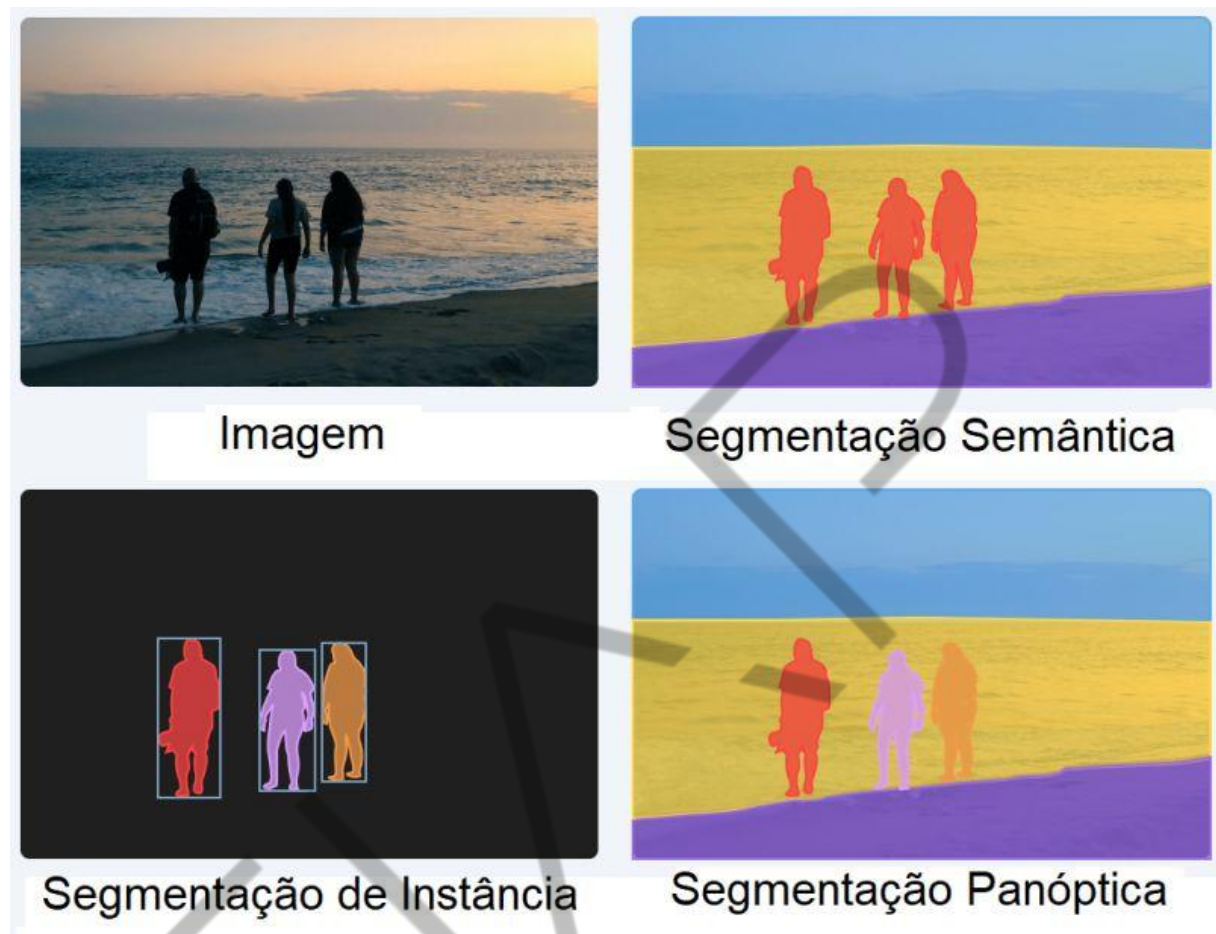


Figura 9 - Comparação entre tipos de segmentação

Fonte: LinkedIn (2023)

Como demonstrado, as diferentes formas de segmentação, semântica, instanciada e panóptica, representam níveis crescentes de detalhamento na compreensão de imagens, indo desde rótulos gerais por pixel até informações completas de classe e identidade para cada elemento visual da cena. Cada uma dessas abordagens possui algoritmos, arquiteturas de redes neurais e métricas de avaliação próprias, sendo escolhidas conforme os requisitos de precisão e granularidade da aplicação.

5 DETECÇÃO E SEGMENTAÇÃO EM VÍDEO

A transição da detecção e segmentação de objetos em imagens estáticas para vídeos e fluxos contínuos adiciona uma dimensão temporal à tarefa, exigindo que os sistemas não apenas identifiquem eventos em cada quadro, mas também preservem consistência e contexto ao longo do tempo. Enquanto em imagens isoladas a detecção ou segmentação pode ser analisada quadro a quadro, em vídeos essa abordagem isolada raramente é suficiente para aplicações que exigem compreensão dinâmica de cena.

Em um fluxo contínuo, objetos se movem, mudam de escala, podem ficar parcialmente ocultos e sofrer distorções por desfoque de movimento (motion blur), o que introduz desafios específicos que não existem em tarefas puramente espaciais. A literatura especializada em rastreamento de objetos em vídeo faz uma distinção clara entre object detection, que trata da identificação de objetos em cada quadro, e object tracking, cuja missão é seguir objetos ao longo de múltiplos quadros, mantendo suas identidades.

5.1 Desafios Específicos de Vídeo

A redundância temporal é um dos primeiros desafios a ser reconhecido ao trabalhar com vídeos. Diferente de uma imagem isolada, um vídeo consiste em centenas ou milhares de quadros sucessivos que muitas vezes contêm informações altamente similares. Processar cada quadro completamente como se fosse independente pode resultar em computação redundante e desperdício de recursos, além de aumentar significativamente o custo computacional. Para mitigar esse custo, técnicas que exploram informações entre quadros adjacentes, como fluxos ópticos (optical flow), podem ser empregadas para estimar movimento e reduzir o número de operações pesadas repetidas.

O custo computacional é outro desafio preponderante. Em vídeos de alta resolução ou gravações capturadas em tempo real, a necessidade de processar dezenas de quadros por segundo impõe requisitos rígidos de desempenho. Algoritmos mais simples podem ser executados em tempo real com hardware limitado, mas

perdem precisão em situações complexas; algoritmos mais precisos, por outro lado, tendem a ser mais lentos e menos adequados a contextos de tempo real. Seja em hardware embarcado ou em servidores de borda, encontrar um equilíbrio entre desempenho e precisão é uma tarefa de engenharia crítica.

O fenômeno conhecido como motion blur ou desfoque de movimento, causado pelo deslocamento rápido de objetos relativa à taxa de quadros da câmera, e as oclusões (occlusions), em que objetos se escondem parcial ou completamente atrás de outros, representam desafios adicionais que testam a robustez de qualquer sistema de detecção ou rastreamento em vídeo. Esses efeitos alteram a aparência visual dos objetos de forma significativa entre dois quadros adjacentes, prejudicando tanto a detecção confiável quanto a associação correta das instâncias ao longo do tempo.

5.2 Abordagens para Vídeo

Uma abordagem elementar e frequentemente usada em implementações iniciais é o processamento frame a frame, em que cada quadro é tratado como uma imagem estática independente. Nessa abordagem, métodos de detecção de objetos como YOLO, SSD ou Faster R-CNN são executados para cada quadro, produzindo resultados de detecção que, isoladamente, não garantem continuidade temporal. Essa técnica é simples de implementar, mas ignora a dependência temporal que é intrínseca a vídeos, resultando em possíveis inconsistências quando um objeto desaparece momentaneamente devido a oclusões ou desfoque.

Outro conceito amplamente utilizado em Visão Computacional é tracking-by-detection, em que um detector de objetos é empregado para identificar possíveis instâncias em cada quadro, e um algoritmo de rastreamento é então usado para associar as detecções de um quadro ao quadro seguinte. Essa associação geralmente envolve modelos de movimento (como o filtro de Kalman) e atributos de aparência que ajudam a manter identidades consistentes ao longo do tempo. Embora simples em conceito, essa técnica enfrenta desafios quando ocorrem aparências semelhantes ou oclusões prolongadas.

Optical flow é uma técnica que pode ser empregada em conjunto com detecção em vídeo para estimar a movimentação de pixels entre quadros adjacentes,

fornecendo informações sobre a direção e velocidade da movimentação de objetos sem realizar uma detecção completa a cada quadro. Isso pode reduzir a necessidade de computar detecções completas em quadros próximos, economizando recursos. Implementações clássicas de fluxo óptico em Python são frequentemente feitas com a biblioteca OpenCV, que oferece métodos como Lucas-Kanade e Farneback para estimar movimento.

Algumas abordagens avançadas tentam incorporar modelos temporais explícitos, como redes recorrentes (RNN, LSTM) ou transformadores aplicados a sequências de quadros, para modelar a evolução de objetos ao longo do tempo, agregando informações contextuais que vão além da simples associação baseado em detecções independentes. Embora esse campo esteja em rápida evolução, soluções desse tipo são mais complexas e exigem conjuntos de dados anotados adequados para treinamento temporal, como benchmarks de tracking de longo prazo.

5.3 Rastreamento de Objetos

O rastreamento de objetos em vídeo é uma área relacionada, mas distinta da detecção. Enquanto a detecção busca identificar e localizar objetos em cada quadro, o rastreamento busca manter uma identidade consistente de cada objeto conforme ele se move ao longo de múltiplos quadros. Ferramentas e algoritmos de rastreamento operam sobretudo no paradigma tracking-by-detection, em que um detector fornece as posições iniciais e um rastreador associa essas posições ao longo do tempo.

O algoritmo SORT (Simple Online and Realtime Tracking) é um método de rastreamento de múltiplos objetos que combina detecção com um modelo de movimento (usualmente um filtro de Kalman) para prever posições futuras dos objetos e associar detecções em quadros sucessivos com base em similaridade espacial e temporal. Suas características de simplicidade e eficiência o tornam popular em aplicações que exigem rastreamento em tempo real, embora ele não lide bem com mudanças drásticas de aparência ou oclusões prolongadas.

Uma evolução importante do SORT é o algoritmo Deep SORT (Deep Simple Online and Realtime Tracking), que adiciona um modelo de aparência baseado em aprendizagem profunda para melhorar a associação entre detecções ao longo do

tempo, reduzindo trocas de identidade (identity switches) e melhorando a estabilidade sob oclusões e mudanças de ângulo. O Deep SORT extrai embeddings de aparência para cada objeto detectado e usa essas características em conjunto com a estimativa de movimento para associar de forma mais robusta as instâncias de objeto entre quadros.

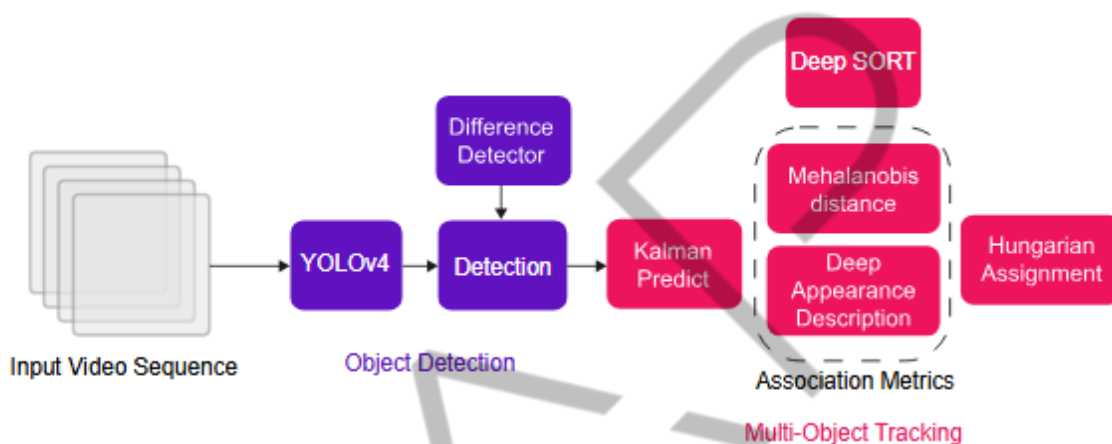


Figura 10 - Algoritmo DeepSort

Fonte: Real Time Pear Fruit Detection and Counting Using YOLOv4 Models and Deep SORT (2021), adaptado por FIAP (2026)

A integração de detectores de objetos modernos com esses algoritmos de rastreamento constitui o que se chama de pipeline de rastreamento, em que um detector como YOLO ou SSD alimenta as posições de objeto para o rastreador, e o rastreador segue esses objetos ao longo do vídeo. Essa integração é fundamental para aplicações em que se deseja monitorar a trajetória de objetos ao longo do tempo, como em vigilância, esportes ou análise de tráfego.

Em Python, é possível implementar um pipeline simples de tracking-by-detection utilizando bibliotecas como OpenCV para leitura de vídeo e frameworks de detecção como YOLOv8 combinados com Deep SORT para manter identidades. Um exemplo resumido de leitura de vídeo e uso de detecção a cada quadro com OpenCV seria:

```
1 | !pip install ultralytics
2 |
3 | import cv2
4 | from ultralytics import YOLO
5 |
6 | # Public raw GitHub video URL
7 | video_url =
"https://raw.githubusercontent.com/harsh2912/people-
tracking/master/videos/MOT16-03.mp4"
8 |
9 | # Load the YOLOv8 model
10 | model = YOLO("yolov8n.pt")
11 |
12 | # OpenCV VideoCapture
13 | cap = cv2.VideoCapture(video_url)
14 | if not cap.isOpened():
15 |     print("Error: cannot open video.")
16 |     exit()
17 |
18 | # Get input video properties
19 | fps = cap.get(cv2.CAP_PROP_FPS) or 30 #
Frames per second (default to 30 if missing)
20 | width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)) #
Frame width
21 | height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)) #
Frame height
22 |
23 | # Choose an output video filename
24 | output_filename = "tracked_output.avi"
25 |
26 | # Define the codec and create VideoWriter
27 | fourcc = cv2.VideoWriter_fourcc(*"XVID") #
Codec for AVI output
28 | out = cv2.VideoWriter(output_filename, fourcc, fps, (width,
height))
29 |
30 | print(f"Saving tracked video to {output_filename} ...")
31 |
32 | while True:
33 |     ret, frame = cap.read()
34 |     if not ret:
35 |         break
36 |
37 |     # Run tracking on the current frame
38 |     results = model.track(frame, persist=True)[0]
39 |
40 |     # Annotated frame with boxes and track IDs
41 |     annotated_frame = results.plot()
42 |
43 |     # Write frame to output video
44 |     out.write(annotated_frame)
```

```
45|  
46| # Release everything  
47| cap.release()  
48| out.release()  
49| print("Done. Video saved.")
```

Código-fonte 3 - Object Tracking
Fonte: Elaborado pelo autor (2025)



Figura 11 - Saída do Algoritmo de Object Tracking para um frame
Fonte: Elaborado pelo autor (2025)

Esse código básico pode ser estendido para incorporar chamadas ao detector e ao rastreador, permitindo acompanhar múltiplos objetos ao longo do vídeo.

5.4 Aplicações em Tempo Real

A detecção e segmentação em vídeos e fluxos contínuos é amplamente empregada em câmeras inteligentes, sistemas equipados com sensores de vídeo capazes de processar diretamente fluxos ao vivo para extrair informações úteis sobre eventos em cena. Essas aplicações incluem vigilância urbana inteligente, monitoramento de tráfego, sistemas de alerta automático e análises comportamentais do ambiente.

Em cenários de monitoramento contínuo, como segurança de grandes instalações ou centros urbanos, é fundamental manter a eficiência computacional para que análises de múltiplos feeds de vídeo possam ser realizadas simultaneamente sem perda de desempenho. Métodos que combinam detecção de objetos com

rastreamento eficiente permitem reduzir a quantidade de processamento redundante, melhorando a escalabilidade das soluções.

Sistemas embarcados, como câmeras inteligentes ou dispositivos de borda, exigem otimizações adicionais de hardware e software porque dispõem de recursos limitados. Nesses contextos, modelos de detecção mais leves e estratégias de rastreamento eficientes são preferidos, muitas vezes em conjunto com técnicas de compressão de rede ou inferência acelerada.



6 SEGMENT ANYTHING MODEL 2 (SAM 2): UMA NOVA GERAÇÃO DE MODELOS DE SEGMENTAÇÃO

O Segment Anything Model 2 (SAM 2) é um dos mais recentes avanços na área de segmentação de objetos em Visão Computacional, projetado para oferecer capacidades de segmentação promptable em imagens e vídeos por meio de uma arquitetura única, flexível e eficiente. Ele representa a evolução do projeto original Segment Anything Model (SAM) desenvolvido pela Meta AI, expandindo a capacidade de segmentar qualquer objeto, não apenas em imagens estáticas, mas também em sequências de vídeo em tempo real com memória temporal integrada.

O SAM 2 foi concebido como um modelo foundation de segmentação visual, ou seja, um modelo treinado em larga escala para realizar tarefas de segmentação em múltiplos domínios e contextos sem necessidade de ajustes extensivos para cada nova tarefa. Diferentemente do SAM original, que se concentrava apenas em imagens estáticas, o SAM 2 incorpora componentes que permitem processamento contínuo de vídeos, incluindo mecanismos de memória que preservam informações de objetos ao longo de múltiplos quadros, o que é fundamental para manter a consistência de máscaras em sequência temporal.

Um dos grandes diferenciais do SAM 2 é sua natureza promptable (baseada em prompts): o modelo pode receber indicações simples, como boxes, pontos ou máscaras parciais fornecidos pelo usuário, e produzir máscaras de segmentação precisas para os objetos de interesse com base nesses sinais. Essa abordagem permite que o modelo seja altamente interativo e flexível, facilitando aplicações em edição de imagens, vídeos e criação de dados anotados para treinamento de outros modelos de Visão Computacional.

A arquitetura do SAM 2 é inspirada nas tendências modernas de modelos de visão baseados em transformers, que combinam um codificador de imagem capaz de extrair representações robustas dos quadros com um codificador de prompts que integra informações de entrada do usuário, e um decodificador de máscara que produz as máscaras segmentadas de forma eficiente e em alta velocidade. Essa combinação permite ao modelo lidar com imagens e vídeos complexos com precisão e desempenho adequados para aplicações em tempo real.

Além disso, o SAM 2 foi treinado com conjuntos de dados massivos que incluem imagens e vídeos anotados, produzindo um modelo capaz de realizar segmentação zero-shot, ou seja, segmentar objetos que não estiveram presentes explicitamente no conjunto de treinamento, com bom desempenho geral. Essa capacidade de generalização o torna útil para aplicações que lidam com grande diversidade de cenários visuais.

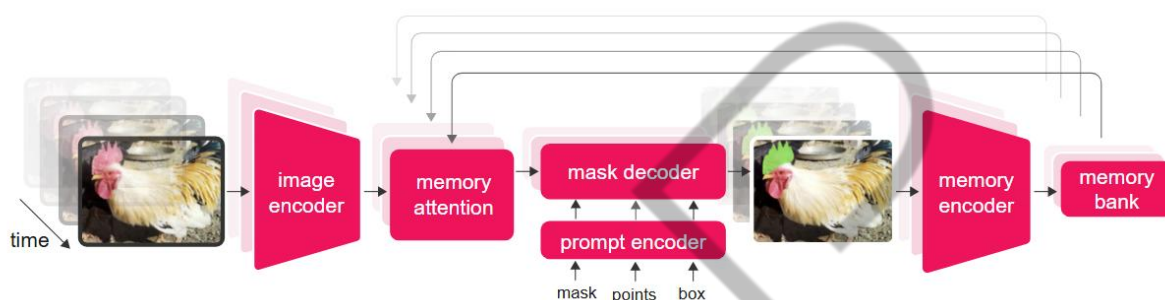


Figura 12 - Algoritmo SAM2
Fonte: Facebook Research (2024), adaptado por FIAP (2026)

Na prática, um usuário pode interagir com o SAM 2 fornecendo um prompt simples para iniciar a segmentação. Por exemplo, dado um ponto ou caixa delimitadora que define uma região de interesse na imagem, o modelo pode gerar automaticamente a máscara de segmentação associada. Essa característica “prompt-to-mask” transforma tarefas que historicamente exigiam segmentação manual em processos automatizados com alta qualidade visual.

Além de aplicações em imagens, o SAM 2 também se destaca no domínio de segmentação de vídeo em tempo real, graças à sua capacidade de incorporar informações temporais entre quadros. Ao utilizar uma memória interna que guarda características relevantes de objetos ao longo dos quadros, o SAM 2 consegue manter a consistência das máscaras mesmo quando objetos se movem, são parcialmente ocultados ou mudam de forma ao longo de uma sequência de vídeo.

Do ponto de vista de implementação prática, embora o uso do modelo requer ferramentas modernas como PyTorch ou frameworks compatíveis com arquitetura transformer e aceleração por GPU, a comunidade já oferece kits de desenvolvimento e interfaces pré-construídas que facilitam a aplicação do SAM 2 para segmentação em workflows de pesquisa e produção. Esses toolkits variam de interfaces em Python que executam inferência em imagens e vídeos a APIs que permitem ajustar prompts dinamicamente e extrair máscaras em tempo real.

Em termos de aplicações, a versatilidade do SAM 2 o torna adequado para cenários como edição de conteúdo visual, onde criadores podem selecionar e modificar objetos de forma precisa e interativa em fotos ou vídeos; construção automatizada de datasets anotados, reduzindo drasticamente o tempo de rotulagem manual; e sistemas autônomos e robóticos avançados, nos quais a segmentação consistente ao longo de sequências de vídeo é crucial para rastreamento, navegação e entendimento de cenas dinâmicas.

Laboratório Prático: SAM2

Antes de implementar o SAM2 no Python, vamos precisar configurar o ambiente do Google Colab e instalar as dependências:

```
1 | import os
2 | HOME = os.getcwd()
3 | print("HOME:", HOME)
4 |
5 | !git clone https://github.com/facebookresearch/segment-
anything-2.git
6 | %cd {HOME}/segment-anything-2
7 | !pip install -e . -q
8 |
9 | !pip install -q supervision jupyter_bbox_widget
10|
11| !mkdir -p {HOME}/checkpoints
12|
13| !wget -q
https://dl.fbaipublicfiles.com/segment_anything_2/072824/sam
2_hiera_large.pt -P {HOME}/checkpoints
```

Código-fonte 4 - Setup do SAM2
Fonte: Elaborado pelo autor (2025)

Em seguida, utilizaremos o SAM2 para segmentar uma imagem de exemplo:

```
1 | import cv2
2 | import torch
3 | import base64
4 |
5 | import numpy as np
6 | import supervision as sv
7 |
8 | from sam2.build_sam import build_sam2
9 | from sam2.sam2_image_predictor import SAM2ImagePredictor
10| from sam2.automatic_mask_generator import
SAM2AutomaticMaskGenerator
11|
```

```
12| torch.autocast(device_type="cuda",
13| dtype=torch.bfloat16).__enter__()
14|
15| if torch.cuda.get_device_properties(0).major >= 8:
16|     torch.backends.cuda.matmul.allow_tf32 = True
17|     torch.backends.cudnn.allow_tf32 = True
18|
19| DEVICE = torch.device('cuda' if
20| torch.cuda.is_available() else 'cpu')
21| CHECKPOINT = f"{HOME}/checkpoints/sam2_hiera_large.pt"
22| CONFIG = "sam2_hiera_l.yaml"
23|
24| sam2_model = build_sam2(CONFIG, CHECKPOINT,
25| device=DEVICE, apply_postprocessing=False)
26|
27| mask_generator = SAM2AutomaticMaskGenerator(sam2_model)
28|
29| IMAGE_PATH = f"{HOME}/carro-completo.jpeg"
30|
31| image_bgr = cv2.imread(IMAGE_PATH)
32| image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
33|
34| sam2_result = mask_generator.generate(image_rgb)
35|
36| mask_annotator =
37| sv.MaskAnnotator(color_lookup=sv.ColorLookup.INDEX)
38| detections =
39| sv.Detections.from_sam(sam_result=sam2_result)
40|
41| annotated_image =
42| mask_annotator.annotate(scene=image_bgr.copy(),
43| detections=detections)
```

Código-fonte 5 - Segmentação de imagem com SAM2
Fonte: Elaborado pelo autor (2025)

A seguir, temos a imagem original e o resultado após a segmentação.



Figura 13 - Saída do algoritmo SAM2
Fonte: Elaborado pelo autor (2025)

Outra possibilidade é analisar as máscaras encontradas pelo algoritmo, utilizando o seguinte trecho de código:

```
1 | masks = [  
2 |     mask['segmentation']  
3 |     for mask  
4 |     in sorted(sam2_result, key=lambda x: x['area'],  
reverse=True)  
5 | ]  
6 |  
7 | sv.plot_images_grid(  
8 |     images=masks[:25],  
9 |     grid_size=(5, 5),  
10 |     size=(12, 12)  
11 | )
```

Código-fonte 6 - Máscaras encontradas pelo SAM2
Fonte: Elaborado pelo autor (2025)



Figura 14 - Máscaras resultantes do SAM2
Fonte: Elaborado pelo autor (2025)

7 LIMITAÇÕES, DESAFIOS ÉTICOS E TENDÊNCIAS FUTURAS

Ao longo deste capítulo, vimos que os modelos de detecção e segmentação de objetos alcançaram níveis impressionantes de desempenho graças ao uso de arquiteturas profundas e grandes volumes de dados anotados. Apesar desses avanços, existem **limitações técnicas fundamentais** que impactam diretamente a eficácia, a robustez e a adoção desses sistemas em contextos do mundo real. Além disso, o uso de tecnologias de Visão Computacional levanta **questões éticas e sociais importantes** que precisam ser discutidas e reguladas. Por fim, o campo está em rápida evolução, com tendências emergentes como modelos foundation, segmentação *zero-shot* e integração multimodal entre visão e linguagem, que prometem moldar o futuro da área.

7.1 Limitações Técnicas

Uma das principais limitações técnicas dos sistemas de detecção e segmentação de objetos é a sua **sensibilidade a variações nas condições de iluminação**. Modelos baseados em redes neurais profundas aprendem padrões visuais a partir de exemplos anotados, mas mudanças abruptas na iluminação de uma cena, como sombras intensas, luz direta do sol ou iluminação baixa, podem degradar significativamente a precisão das predições. Isso ocorre porque a aparência visual dos objetos muda sob diferentes condições de luz, e os modelos muitas vezes não veem exemplos suficientemente variados durante o treinamento para generalizar bem. Para mitigar esse problema, técnicas de data augmentation, como ajustes de brilho e contraste, são aplicadas durante o treinamento para tornar o modelo mais robusto a tais variações.

Outra limitação relevante está relacionada ao viés de dados nos conjuntos de treinamento. Modelos de Visão Computacional dependem de grandes bancos de dados anotados para aprender a reconhecer e segmentar objetos. Se esses dados forem desequilibrados ou não refletirem a diversidade do mundo real, por exemplo, concentrando-se em determinadas populações, cenários ou condições ambientais, os modelos podem performar mal em contextos sub-representados ou até perpetuar

vieses indesejados. Isso impacta a acurácia e a confiança que se pode depositar nas predições em aplicações sensíveis, como saúde ou segurança pública.

Uma terceira limitação técnica refere-se à generalização limitada dos modelos atuais. Embora redes profundas consigam aprender representações visuais complexas, muitos modelos treinados em um conjunto de dados específico, como COCO ou Pascal VOC, tendem a perder desempenho quando aplicados em cenários diferentes daqueles vistos durante o treinamento. Isso pode ocorrer quando há diferenças de contexto, ângulos de câmera, tipos de objetos ou ruídos visuais, exigindo frequentemente esforço adicional de fine-tuning ou adaptação de domínio (domain adaptation) para manter resultados aceitáveis.

Essas limitações técnicas não apenas colocam desafios ao desempenho dos modelos, mas também orientam a pesquisa contínua por soluções mais robustas e generalizáveis.

7.2 Questões Éticas

A aplicação de algoritmos de Visão Computacional em contextos reais também levanta um conjunto complexo de questões éticas que vão além da pura performance técnica. Uma das preocupações mais críticas é **a privacidade dos indivíduos afetados pelas tecnologias**. Sistemas de vigilância baseados em vídeo que utilizam detecção de objetos ou reconhecimento facial podem capturar imagens de pessoas sem seu consentimento explícito, potencialmente criando bancos de dados visuais sensíveis sem controle individual. A utilização dessas tecnologias sem regulamentação adequada pode comprometer direitos fundamentais de privacidade e liberdade.

A questão da vigilância em massa está diretamente relacionada à privacidade. Em muitos países, câmeras públicas e privadas vêm sendo equipadas com sistemas automáticos de detecção e rastreamento de pessoas, veiculando preocupações sobre quem monitora os dados, como eles são armazenados e para que fins são utilizados. Isso pode levar a cenários em que a tecnologia seja usada para controle social ou discriminação, em vez de prover segurança de forma ética e transparente.

Outro ponto crucial é o **uso responsável da tecnologia** de Visão Computacional. A adoção de modelos de detecção e segmentação deve levar em consideração a precisão dos resultados e as consequências de decisões automatizadas. Por exemplo, em sistemas de diagnóstico médico assistido por IA, decisões errôneas podem afetar diretamente a saúde de pacientes, enquanto em veículos autônomos, falhas de segmentação podem resultar em acidentes. A pesquisa em IA responsável enfatiza que os modelos devem ser auditáveis, interpretáveis e estar sujeitos a mecanismos de governança que filtrem vieses indesejados.

Essas questões éticas exigem um diálogo contínuo entre pesquisadores, legisladores, desenvolvedores e o público para criar frameworks legais e técnicas que equilibrem inovação tecnológica com proteção de direitos individuais.

7.3 Tendências Futuras

O futuro da detecção e segmentação de objetos aponta para modelos mais gerais e adaptáveis, frequentemente chamados de **modelos foundation** para visão. Assim como os grandes modelos de linguagem que deram origem a sistemas como GPT, esses modelos de base são treinados com enormes quantidades de dados visuais e podem ser adaptados a múltiplas tarefas de Visão Computacional com pouco ajuste adicional, promovendo maior reutilização e eficiência.

Outro campo emergente é o da **segmentação zero-shot**, que busca permitir que modelos reconheçam e segmentem categorias de objetos que não foram explicitamente vistas durante o treinamento. Essa capacidade de generalizar para novas classes de forma automática reduziria a dependência de conjuntos de dados anotados extensivamente e permitiria aplicações mais flexíveis em ambientes dinâmicos. Pesquisas recentes nessa área revelam desafios significativos em manter precisão e robustez em tarefas zero-shot, mas também mostram o potencial de oferecer sistemas mais adaptáveis.

Por fim, uma das tendências mais fortes na pesquisa atual é a **multimodalidade**, ou integração entre visão e linguagem, que combina capacidades visuais com compreensão de texto ou linguagem natural. Modelos vision-language (VLMs) capacitam sistemas a interpretar imagens ou vídeos com contexto semântico

adicional, como responder perguntas sobre uma cena visual ou gerar descrições textuais de situações complexas. Essa integração amplia as capacidades dos sistemas de IA além da mera percepção visual para uma compreensão contextual mais rica, e está sendo explorada ativamente em workshops e pesquisas do campo.

Essas tendências apontam para um futuro em que sistemas de Visão Computacional não apenas sejam mais poderosos e versáteis, mas também mais alinhados com necessidades humanas complexas, incluindo comunicação multimodal e adaptação a cenários não antecipados.



CONCLUSÃO

Ao longo deste capítulo, exploramos de forma integrada os principais conceitos, métodos e aplicações de detecção e segmentação de objetos, que são tarefas centrais da Visão Computacional moderna. Discursos introdutórios estabeleceram as diferenças entre classificação de imagens, detecção de objetos e diferentes formas de segmentação, semântica, instanciada e panóptica, e mostraram como cada uma delas contribui para que os computadores interpretem o mundo visual de maneira mais refinada e precisa. Em seguida, examinamos modelos e arquiteturas avançadas, tanto de duas etapas quanto de uma etapa, como Faster R-CNN, YOLO e SSD, e discutimos suas características, vantagens e limitações, ilustrando o equilíbrio entre precisão e desempenho exigido em aplicações reais. Também foram abordadas as métricas matemáticas que sustentam a avaliação rigorosa de detectores e segmentadores, incluindo medidas de sobreposição, precisão e recall, essenciais para comparar e validar modelos em ambientes diversos.

A parte prática do capítulo trouxe ao leitor experiências concretas de implementação em Python, com orientação para configurar o ambiente de desenvolvimento, treinar modelos em conjuntos de dados como COCO e fine-tunar redes profundas para casos de uso específicos. Projetos sobre detecção em imagens estáticas, segmentação instanciada e aplicações em vídeo em tempo real foram utilizados como laboratórios práticos, permitindo compreender tanto a teoria quanto os desafios encontrados na prática, como desempenho computacional e precisão de predição. Esses elementos práticos consolidam a relevância da Visão Computacional como campo aplicável e em rápida evolução.

A importância da detecção e segmentação para sistemas inteligentes se manifesta em uma ampla variedade de aplicações que exigem percepção automatizada. Em veículos autônomos, por exemplo, a capacidade de localizar pedestres, veículos e obstáculos em tempo real é fundamental para a segurança e operação autônoma confiável. Em saúde, a segmentação precisa de estruturas em imagens médicas auxilia no diagnóstico assistido por IA. Em cenários industriais e de varejo, a Visão Computacional permite automação sofisticada de inspeção, controle de qualidade e análise de comportamento, áreas nas quais a interpretação visual automatizada traduz dados em decisões operacionais automatizadas. Esse papel

crítico está alinhado com o entendimento de que a Visão Computacional permite traduzir dados visuais em informações significativas e acionáveis, que vão além da simples classificação ou detecção, contribuindo para a automação inteligente.

O capítulo também abriu espaços para discutir desafios técnicos e éticos, como viés de dados, sensibilidade a variações de iluminação e preocupações com privacidade, destacando que um entendimento técnico sólido precisa andar junto com uma perspectiva crítica sobre o uso responsável dessas tecnologias no mundo real. Isso prepara para engajar com problemas complexos que vão além de códigos e modelos, integrando aspectos de governança, impacto social e sustentabilidade de sistemas baseados em IA.

Finalmente, esta conclusão articula a conexão desta parte do conteúdo com capítulos futuros. A Visão Computacional, especialmente nos tópicos de detecção e segmentação, está cada vez mais integrada com outras áreas emergentes da Inteligência Artificial. A multimodalidade, que combina visão com linguagem natural, surge como uma fronteira recente que amplia a capacidade de sistemas de IA não apenas identificar e segmentar objetos, mas também interpretá-los e contextualizá-los com base em instruções em linguagem humana ou em descrições semânticas livres, uma direção que está sendo intensamente pesquisada com modelos de linguagem visual (vision-language models) capazes de reconhecimento e inferência zero-shot e raciocínio contextualizado.

Outro rumo promissor para os próximos capítulos é a exploração de agentes visuais ou sistemas multimodais que combinam capacidades de visão com planejamento e tomada de decisão, abrindo espaço para aplicações mais complexas como robôs autônomos e assistentes inteligentes que interpretam o ambiente visual para agir de forma contextualizada. Esses desenvolvimentos não apenas consolidam a visão computacional como pilar da IA aplicada, mas também alinham o aprendizado técnico com tendências emergentes no design de sistemas inteligentes que entendem e interagem com o mundo de forma integrada e adaptativa.

REFERÊNCIAS

BRADSKI, G. The OpenCV Library. **Dr. Dobb's Journal of Software Tools**. [S. l.], 2000.

CANNY, J. A Computational Approach to Edge Detection. **IEEE Transactions on Pattern Analysis and Machine Intelligence**. [S. l.], v. PAMI-8, n. 6, p. 679-698, nov. 1986.

DATA SCIENCE TEAM. Mask R-CNN. **DataScience**, 2019. Disponível em: <<https://datascience.eu/computer-vision/mask-r-cnn/>>. Acesso em: 12 fev. 2026.

DENG, J. *et al.* Construction and Analysis of a Large Scale Image Ontology. *In: Vision Sciences Society (VSS)*, 2009.

DIGITALOCEAN. Faster R-CNN Explained for Object Detection Tasks. **Digital Ocean**, 2025. Disponível em: <<https://www.digitalocean.com/community/tutorials/faster-r-cnn-explained-object-detection>>. Acesso em: 12 fev. 2026.

DOSOVITSKIY, A. *et al.* An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. **arXiv**, 2020. Disponível em: <<https://arxiv.org/abs/2010.11929>>. Acesso em: 11 fev. 2026.

HE, K. *et al.* Deep Residual Learning for Image Recognition. *In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. p. 770-778.

KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. ImageNet classification with deep convolutional neural networks. **Communications of the ACM**, [S. l.], v. 60, n. 6, p. 84–90, 2017.

LECUN, Y.; BENGIO, Y. Convolutional networks for images, speech, and time series. **The handbook of brain theory and neural networks**. [S. l.], v. 3361, n. 10, 1995.

LECUN, Y. *et al.* Gradient-based learning applied to document recognition. **Proceedings of the IEEE**. [S. l.], v. 86, n. 11, p. 2278-2324, nov. 1998.

LOWE, D. G. Distinctive Image Features from Scale-Invariant Keypoints. **International Journal of Computer Vision**. [S. l.], v. 60, n. 2, p. 91–110, nov. 2004.

MASMOUDI, N. *et al.* UAV-based crowd surveillance in post covid-19 era. **IEEE Access**, 2021. Disponível em: <https://www.researchgate.net/publication/356867749_UAV-based_crowd_surveillance_in_post_covid-19_era>. Acesso em: 12 fev. 2026.

PARICO, A. I. B.; AHAMED, Tofael. Real Time Pear Fruit Detection and Counting Using YOLOv4 Models and Deep SORT. **ResearchGate**, 2021. Disponível em: <https://www.researchgate.net/publication/353256407_Real_Time_Pear_Fruit_Detection_and_Counting_Using_YOLOv4_Models_and_Deep_SORT>. Acesso em: 12 fev. 2026.

RAVI, N. *et al.* Sam 2: Segment anything in images and videos. **arXiv**, 2024. Disponível em: <<https://arxiv.org/abs/2408.00714>>. Acesso em: 11 fev. 2026.

REDMON, J. *et al.* You only look once: Unified, real-time object detection. *In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

ROUIZI, Yacine. Non-Maximum Suppression with OpenCV and Python. **The Python Code**, 2024. Disponível em: <<https://thepythoncode.com/article/non-maximum-suppression-using-opencv-in-python>>. Acesso em: 12 fev. 2026.

SIMONYAN, K.; ZISSERMAN, A. Very deep convolutional networks for large-scale image recognition. **arXiv**, 2014. Disponível em: <<https://arxiv.org/abs/1409.1556>>. Acesso em: 11 fev. 2026.

TERVEN, J. *et al.* Loss Functions and Metrics in Deep Learning. A Review. **arXiv**, 2023. Disponível em: <https://arxiv.org/abs/2307.02694>. Acesso em: 12 fev. 2026.

YAJNIK, Ayush. Computer Vision: YOLO introduction. **Medium**, 2023. Disponível em: <<https://medium.com/@ayushyajnik2/computer-vision-yolo-introduction-49c873c1d6a0>>. Acesso em: 12 fev. 2026.